

**UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE POLITEHNICA
BUCUREȘTI**

FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI

PROGRAMUL DE MASTERAT: INGINERIA INFORMAȚIEI ȘI A SISTEMELOR DE CALCUL

PROIECT

Data Warehouse în Clickhouse

Coordonator științific:

S.I. Dr. Ing. Valentin Pupezescu

Student:

Stanciu Răducu-Ștefan
411-IISC

BUCUREȘTI

2026

Cuprins

1	Introducere	3
2	Clickhouse	4
2.1	De ce este ClickHouse atât de rapid?	5
2.2	Caracteristicile distinctive ale ClickHouse	5
2.3	Motorul MergeTree	6
2.3.1	Structura fizică a datelor	6
2.3.2	Cheia primară	7
2.3.3	Partiționarea datelor	7
2.3.4	Data Skipping Indexes	7
2.3.5	TTL și ciclul de viață al datelor	8
2.3.6	Concurență și consistență	8
2.4	Compresia în ClickHouse	8
3	Implementare Practică	9
3.1	Docker	9
3.2	DBeaver	10
3.3	Crearea Tabelei	11
3.4	Analiza performanței ClickHouse	13
3.4.1	Scanare Totală	16
3.4.2	Filtrare Array	16
3.4.3	Serii Temporale	17
3.4.4	Activitate Utilizatori (Cardinalitate Mare)	18
4	Concluzii	19

1. Introducere

Din punct de vedere tehnic, sistemele moderne de gestiune a bazelor de date se împart în două categorii fundamentale: procesarea analitică online (OLAP) și procesarea tranzacțională online (OLTP). Prima fiind specializată în generarea rapoartelor complexe pe baza unor volume masive de date istorice, în timp ce a doua gestionează fluxuri constante de tranzacții ce modifică starea curentă a informației. Deși istoric aceste categorii au fost tratate separat, realitatea tehnologică actuală tinde spre convergență sub forma sistemelor HTAP (Hybrid Transactional/Analytical Processing), care încearcă să echilibreze ambele tipuri de sarcini de lucru într-o singură platformă pentru a reduce costurile de întreținere și complexitatea integrării. ClickHouse se încadrează în acest spectru ca un sistem OLAP proiectat inițial pentru viteză extremă, care, deși nu oferă suport tranzacțional complet, a evoluat prin introducerea unor funcționalități precum mutațiile pentru actualizarea datelor și consistența citirii pentru a răspunde cerințelor moderne. Compromisul tehnic esențial dintre cele două arhitecturi constă în modul de organizare a datelor pe disc, unde sistemele OLAP adoptă un format columnar pentru a permite citirea eficientă și izolată a atributelor necesare analizelor mari, în timp ce sistemele OLTP preferă stocarea pe rânduri pentru a optimiza operațiunile de scriere și modificare individuală, evitând astfel penalitățile de performanță ce apar atunci când fiecare coloană trebuie accesată separat în cazul structurilor foarte late. [1]

Conceptul de Procesare Analitică Online, cunoscut sub acronimul OLAP, reprezintă baza arhitecturii moderne a depozitelor de date, oferind organizațiilor capacitatea de a transforma volume imense de date brute în perspective strategice prin analiză multidimensională. Spre deosebire de sistemele tranzacționale clasice care sunt optimizate pentru scrierea rapidă a datelor individuale, sistemele OLAP sunt proiectate special pentru interogări complexe care implică agregări masive și calcule sintetice. Arhitectura din spatele acestui proces începe cu colectarea informațiilor din surse eterogene, precum aplicații web sau baze de date interne, care sunt ulterior trecute prin procese de tip ETL pentru a fi standardizate și curățate. Odată pregătite, aceste date sunt adesea structurate sub forma unor cuburi OLAP sau hipercuburi, care permit vizualizarea informațiilor pe mai multe axe simultan, cum ar fi produsul, locația geografică și timpul. Această structură multidimensională elimină latența specifică bazelor de date relaționale tradiționale, deoarece majoritatea calculelor sunt precalculate, permițând utilizatorilor non-tehnici să exploreze datele prin operațiuni intuitive precum rotația axelor prin pivotare, restrângerea detaliilor prin roll-up sau investigarea profunzimii informației prin drill-down.

În funcție de necesitățile de stocare și viteză, tehnologia se poate manifesta sub formă de MOLAP, unde datele sunt stocate direct într-un format multidimensional pentru viteză maximă, ROLAP, care analizează datele direct din tabele relaționale folosind scheme de tip stea sau fulg de zăpadă, sau HOLAP, care hibridizează ambele abordări pentru a oferi un echilibru între granulația detaliilor și performanța interogării. Prin integrarea acestor fluxuri, OLAP devine componentă fundamentală pentru Business Intelligence, oferind o viziune unificată asupra întregii afaceri și facilitând procese decizionale rapide, bazate pe dovezi istorice și analize de tip impact, esențiale într-un ecosistem economic global digitalizat și extrem de competitiv. [2]

2. Clickhouse

ClickHouse® reprezintă un sistem de gestiune a bazelor de date SQL de înaltă performanță, special conceput pentru procesarea analitică online (OLAP), fiind capabil să proceseze volume masive de date, de ordinul miliardelor sau trilioanelor de rânduri, în timp real.

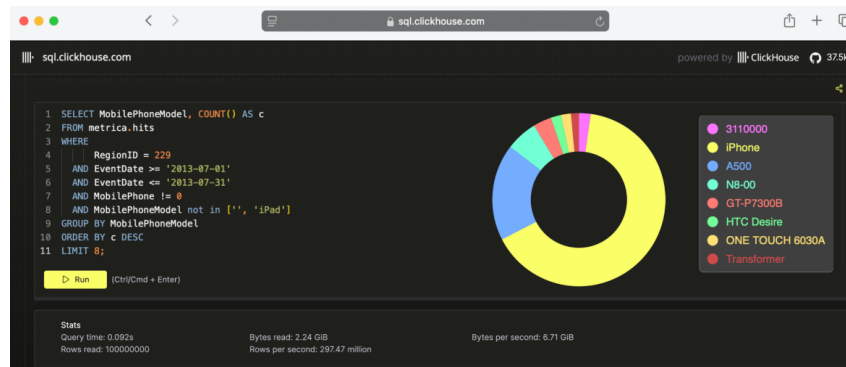


Figura 1: În secțiunea de statistici din diagrama de mai sus, interogarea a procesat 100 de milioane de rânduri în 92 de milisecunde, ceea ce înseamnă un throughput de aproximativ peste 1 miliard de rânduri pe secundă sau puțin sub 7 GB de date transferate pe secundă.

Superioritatea sa tehnică derivă din arhitectura orientată pe coloane, o abordare care stochează valorile fiecărei coloane secvențial pe disc, eliminând astfel citirile inutile de date și optimizând operațiunile de filtrare și agregare prin reducerea drastică a fluxului I/O. Această eficiență este completată de mecanisme avansate precum replicarea asincronă multi-master pentru integritatea datelor, suportul SQL declarativ compatibil cu standardul ANSI și algoritmi adaptivi de join. Mai mult, ClickHouse permite optimizarea resurselor prin calcule aproximative și eșantionare, oferind un control riguros al accesului prin utilizatori și roluri, toate aceste caracteristici transformându-l într-o soluție robustă și scalabilă pentru depozite de date moderne care necesită rezultate instantanee pe seturi de date complexe. [3]

Arhitectura sa se bazează pe un motor de execuție a interogărilor vectorizat, care procesează datele în blocuri de rânduri pentru a minimiza apelurile de funcții virtuale, beneficiind totodată de compilarea oportunistă a codului prin LLVM pentru a fuziona operatorii adiacenți și a utiliza eficient registrele CPU. Un avantaj tehnologic major este utilizarea instrucțiunilor SIMD pentru paralelizarea la nivel de elemente de date, permițând sistemului să ruleze pe hardware divers, de la laptopuri vechi până la servere puternice, prin alegerea dinamică a celui mai rapid kernel de calcul disponibil la runtime. Stratul de stocare este construit în jurul familiei de motoare de tabele MergeTree, care utilizează o structură bazată pe arbori LSM, unde datele sunt organizate în părți imutabile sortate, fuzionate continuu în fundal fără a impacta performanța inserărilor. Eficiența accesului la date este garantată prin tehnici agresive de eliminare (pruning), precum utilizarea indicilor primari care rămân în memoria RAM, a indicilor de tip skipping pentru metadata ușoare și a proiecțiilor care oferă versiuni alternative ale tabelelor sortate diferit.

ClickHouse suportă transformarea incrementală a datelor la momentul fuziunii părților prin strategii diverse, cum ar fi înlocuirea versiunilor vechi ale tuplurilor, agregarea rândurilor cu aceleași chei primare în vizualizări materializate sau aplicarea regulilor TTL pentru arhivarea sau ștergerea automată

a datelor istorice pe stocări mai ieftine, cum ar fi S3. De asemenea, sistemul oferă robustețe industrială prin replicarea multi-master asincronă bazată pe consensul Raft, gestionat de ClickHouse Keeper, ceea ce asigură disponibilitate ridicată și toleranță la defectarea nodurilor. Flexibilitatea integrării este un alt punct forte, baza de date dispunând de peste 50 de funcții și motoare de integrare pentru conectarea cu sisteme externe precum MySQL, PostgreSQL, Kafka sau baze de date de tip S3, suportând totodată peste 90 de formate de date, inclusiv JSON, Parquet și Avro. Din punct de vedere al utilizabilității, ClickHouse oferă un dialect SQL bogat în funcționalități, suport pentru controlul accesului bazat pe roluri (RBAC) și instrumente sofisticate de introspecție a performanței, cum ar fi profilerul de eșantionare și integrarea cu OpenTelemetry. Capacitatea de a scala este demonstrată prin sharding-ul transparent al datelor între mai multe noduri, permițând procesarea seturilor de date ce depășesc capacitatea unui singur server prin distribuirea sarcinii de citire și scriere. [4]

2.1. De ce este ClickHouse atât de rapid?

Performanța extremă a sistemului ClickHouse se datorează unor inovații majore implementate atât la nivelul stratului de stocare, cât și în cel de procesare a interogărilor. În stratul de stocare, fiecare tabelă este divizată în mai multe părți, create la fiecare inserție, proces care permite scrieri extrem de rapide, aproape la viteza I/O a discului, deoarece inserțiile simultane nu necesită sincronizare globală. Pentru a gestiona eficiența, ClickHouse rulează în fundal operațiuni de fuziune (merges) care combină părțile mici în structuri mai mari, ocazie cu care sunt executate și transformări de date esențiale, precum eliminarea versiunilor vechi, agregarea incrementală sau aplicarea regulilor de expirare (TTL). Aceste procese de fundal degreuează timpul de execuție al utilizatorului, oferind interogări de până la 1000 de ori mai rapide prin utilizarea datelor deja transformate.

Eficiența este sporită de tehnici avansate de „data pruning”, precum indicii de cheie primară care permit căutări binare rapide, proiecțiile de tabele sortate diferit și indicii de tip skipping care stochează statistici suplimentare pentru a evita citirea rândurilor irelevante. De asemenea, orientarea pe coloane permite o compresie superioară a datelor folosind codec-uri specializate, reducând atât spațiul ocupat, cât și limitările de lățime de bandă ale discurilor. La nivelul procesării, ClickHouse utilizează un model vectorizat unde datele sunt transmise în loturi între operatori, permițând utilizarea instrucțiunilor SIMD și paralelizarea execuției pe toate nucleele CPU disponibile sau chiar pe mai multe noduri într-un cluster.

Dincolo de arhitectură, succesul sistemului constă într-o atenție meticuloasă la detaliile de optimizare de nivel scăzut, utilizând, de exemplu, peste 30 de variante specializate de tabele hash și algoritmi care se adaptează dinamic la caracteristicile datelor în timpul execuției. Toate aceste componente sunt integrate într-un binar nativ C++, fără dependențe externe, optimizat pentru a obține performanțe maxime direct pe hardware, ceea ce plasează ClickHouse printre cele mai rapide baze de date analitice de pe piață. [5]

2.2. Caracteristicile distinctive ale ClickHouse

ClickHouse se distinge ca un sistem de gestiune a bazelor de date veritabil orientat pe coloane, optimizat pentru procesarea analitică prin stocarea compactă a datelor, fără metadata inutile atașate valo-

rilor individuale. Această arhitectură permite atingerea unor viteze de procesare de sute de milioane de rânduri pe secundă, depășind net sistemele care, deși pot stoca coloane separat, sunt optimizate pentru alte scenarii de utilizare. Un element central al performanței sale este motorul de calcul vectorial, care procesează datele în fragmente de coloane (vectori), maximizând eficiența utilizării procesorului.

Spre deosebire de alte sisteme OLAP care necesită bugete hardware imense pentru a rula exclusiv în RAM, ClickHouse este proiectat să funcționeze eficient pe hard disk-uri obișnuite, utilizând în același timp la maximum unitățile SSD și memoria RAM disponibilă. Compresia datelor joacă un rol critic, oferind atât codec-uri de uz general, cât și specializate pentru diverse tipuri de date, ceea ce îi permite să concureze cu succes împotriva bazelor de date de nișă, precum cele pentru serii temporale. Inserarea datelor se realizează în timp real, fără blocaje (locks), utilizând structuri de tip „merge tree” pentru sortarea incrementală, ceea ce permite adăugarea continuă de informații noi în tabele.

Capacitățile de interogare sunt susținute de un limbaj SQL declarativ, compatibil în mare parte cu standardul ANSI, oferind suport pentru operațiuni complexe precum GROUP BY, ORDER BY, JOIN sau funcții fereastră. ClickHouse excelează în furnizarea de rezultate cu latență scăzută, fiind capabil să genereze rapoarte în mod „online” chiar în momentul încărcării unei pagini web, fără a necesita pregătirea prealabilă a răspunsurilor. Pentru optimizarea resurselor, sistemul permite calcule aproximative prin funcții de agregare specifice, eșantionarea datelor (SAMPLE) sau limitarea numărului de chei procesate, oferind utilizatorului control asupra compromisului dintre acuratețe și viteză.

Distribuirea și scalabilitatea sunt asigurate prin suportul nativ pentru procesarea pe mai multe servere, unde datele pot fi la bază diferite shard-uri procesate în paralel și transparent pentru utilizator. Integritatea și disponibilitatea datelor sunt garantate prin replicarea asincronă multi-master, sistemul fiind capabil să recupereze automat datele după majoritatea defecțiunilor. În plus, ClickHouse integrează un sistem robust de control al accesului bazat pe roluri (RBAC), similar cu cel din bazele de date relaționale populare, permițând gestionarea securizată a conturilor de utilizator direct prin interogări SQL. [6]

2.3. Motorul MergeTree

Motorul MergeTree din ClickHouse este cel mai utilizat și cel mai robust motor de stocare pentru tabele analitice. El este proiectat pentru volume foarte mari de date și pentru rate ridicate de ingestie, fiind optimizat pentru scenarii OLAP (Online Analytical Processing). Atunci când sunt inserate date, acestea nu sunt adăugate direct într-o structură unică, ci sunt scrise sub formă de părți independente (data parts). Ulterior, un proces de fundal le îmbină progresiv pentru a optimiza stocarea și performanța interogărilor. Acest mecanism permite scrieri rapide și citiri eficiente în paralel, fără blocaje majore între operațiile de inserare și cele de selecție.

2.3.1 Structura fizică a datelor

Fiecare inserare generează o parte nouă, care este sortată lexicografic după cheia primară definită prin ORDER BY. Datele sunt organizate fizic în aceste părți, iar părțile aparținând unor partiții diferite nu sunt niciodată îmbinate între ele. În fundal, ClickHouse execută operații de merge pentru a combina părți mai mici în părți mai mari, reducând fragmentarea și îmbunătățind eficiența citirilor.

La nivel intern, fiecare parte este împărțită în unități numite granule. O granule reprezintă cel mai mic set de date care poate fi citit de pe disc și conține un număr fix sau aproape fix de rânduri (implicit aproximativ 8192). Pentru fiecare granule, sistemul stochează marcaje (marks) într-un fișier de index. Acest mecanism permite accesul rapid la blocuri de date fără a scana întregul tabel.

2.3.2 Cheia primară

În MergeTree, cheia primară nu funcționează ca într-o bază de date tranzacțională clasică. Ea nu garantează unicitatea și nu indexează fiecare rând individual. În schimb, determină ordinea fizică a datelor în interiorul fiecărei părți și construiește un index rar bazat pe granule.

Indexul conține doar informații despre începutul fiecărei granule, nu despre fiecare rând. Acest lucru îl face foarte compact și suficient de mic pentru a rămâne în memoria RAM chiar și pentru seturi de date foarte mari. Atunci când o interogare conține condiții pe coloanele din cheia primară, sistemul poate identifica rapid intervalele relevante de granule și poate evita citirea inutilă a altor segmente de date.

Este important de reținut că performanța maximă se obține atunci când condițiile din interogări folosesc prefixul cheii primare. De exemplu, dacă cheia este (A, B), o condiție pe A sau pe (A, B) va fi eficientă, în timp ce o condiție doar pe B nu va beneficia complet de index.

2.3.3 Partiționarea datelor

MergeTree permite împărțirea logică a datelor în partiții folosind expresia PARTITION BY. Partiționarea este utilă mai ales pentru date temporale, de exemplu partiționarea lunară cu funcția toYYYYMM(date). În acest caz, fiecare lună devine o partiție separată.

Partiționarea nu accelerează direct interogările prin indexare, dar permite eliminarea completă a partițiilor nerelevante înainte de citire, proces numit partition pruning. Dacă o interogare solicită doar date dintr-o anumită perioadă, ClickHouse poate ignora complet partițiile care nu corespund intervalului respectiv. Totuși, o partiționare prea granulară (de exemplu per utilizator sau per ID) poate duce la fragmentare excesivă și la degradarea performanței.

2.3.4 Data Skipping Indexes

Pe lângă indexul primar, MergeTree suportă indexuri suplimentare de tip „data skipping”. Acestea nu stochează poziții exacte ale rândurilor, ci informații agregate despre blocuri de date, cum ar fi valorile minime și maxime (MinMax), seturi de valori distincte sau filtre Bloom.

Scopul lor este de a reduce volumul de date citit de pe disc prin eliminarea blocurilor care sigur nu conțin valori relevante pentru condiția din interogare. De exemplu, un index MinMax poate permite ignorarea completă a unor blocuri dacă intervalul lor de valori nu intersectează intervalul căutat.

Este important de menționat că unele tipuri de index, precum Bloom filter, pot produce rezultate fals pozitive, ceea ce înseamnă că uneori vor fi citite mai multe date decât strict necesar, dar niciodată mai puține decât trebuie pentru a asigura corectitudinea rezultatului.

2.3.5 TTL și ciclul de viață al datelor

MergeTree permite definirea unor reguli TTL (Time To Live) atât la nivel de coloană, cât și la nivel de tabel. Aceste reguli stabilesc cât timp sunt păstrate datele și ce se întâmplă când expiră. Datele pot fi șterse automat, mutate pe alte volume de stocare sau chiar agregate.

Procesul de eliminare sau mutare a datelor expirate are loc în timpul operațiilor de merge din fundal. Astfel, nu este necesar un proces separat de curățare. Această funcționalitate este extrem de utilă în aplicații cu date temporale, cum ar fi loguri, evenimente sau date IoT, unde retenția este limitată la un anumit interval de timp.

Politici de stocare și utilizarea mai multor disk-uri

MergeTree suportă politici avansate de stocare care permit distribuirea datelor pe mai multe disk-uri sau volume. De exemplu, datele recente („hot data”) pot fi păstrate pe SSD-uri rapide, iar datele istorice („cold data”) pot fi mutate pe HDD-uri sau pe stocare obiect, cum ar fi S3.

Sistemul poate muta automat părțile între volume în funcție de dimensiune sau de nivelul de ocupare a spațiului. Această arhitectură pe niveluri permite optimizarea costurilor fără a sacrifica performanța pentru datele accesate frecvent.

2.3.6 Concurență și consistență

MergeTree utilizează un model de multi-versioning pentru a permite citiri și scrieri simultane fără blocaje majore. Atunci când o interogare SELECT este executată, ea vede o versiune stabilă a setului de părți active la momentul respectiv. Inserările ulterioare nu afectează rezultatul interogării deja pornite.

Citirile sunt automat paralelizate, ceea ce înseamnă că sistemul poate folosi mai multe thread-uri pentru a scana diferite părți sau granule simultan. Această caracteristică contribuie semnificativ la performanța ridicată a interogărilor analitice pe volume mari de date. [7]

2.4. Compresia în ClickHouse

În ClickHouse, compresia este un element central pentru performanță. Deoarece sistemul este columnar, valorile similare sunt stocate una lângă alta, ceea ce permite algoritmilor de compresie să obțină rate foarte bune de reducere a dimensiunii datelor. O dimensiune mai mică pe disc înseamnă mai puțin I/O și interogări mai rapide.

Există două niveluri principale implicate în optimizare: alegerea tipurilor de date și alegerea algoritmilor de compresie (codec-uri). Tipurile de date corecte reduc dimensiunea necomprimată în memorie, iar codec-urile reduc dimensiunea fizică pe disc. În plus, cheia de ordonare influențează compresia, deoarece grupează valori apropiate sau repetate.

Cele mai importante algoritmi de compresie sunt LZ4 și ZSTD. LZ4 este foarte rapid la compresie și decompresie, având un consum redus de CPU. Este potrivit pentru scenarii unde viteza este prioritară și datele sunt citite frecvent. În schimb, ZSTD oferă de obicei o rată de compresie mai bună, reducând mai mult spațiul ocupat pe disc, dar cu un cost ușor mai mare la inserare. [8]

3. Implementare Practică

3.1. Docker

În prima etapă a fost necesară instalarea aplicației Docker Desktop pentru Windows. Docker permite rularea aplicațiilor în containere izolate, fără a instala direct serviciul în sistemul de operare. În cazul de față, ClickHouse a fost rulat într-un container oficial bazat pe imaginea furnizată de ClickHouse prin Docker Hub.

După instalarea Docker, s-a descărcat imaginea oficială folosind comanda de tip „docker run”, care are și rolul de a crea containerul. Prin această comandă au fost expuse porturile 8123 și 9000. Portul 8123 este utilizat pentru interfața HTTP (acces prin browser), iar portul 9000 este folosit pentru conexiuni native ClickHouse. De asemenea, a fost setată limita pentru numărul de fișiere deschise, deoarece ClickHouse lucrează cu multe fișiere interne și necesită o valoare ridicată pentru performanță optimă.

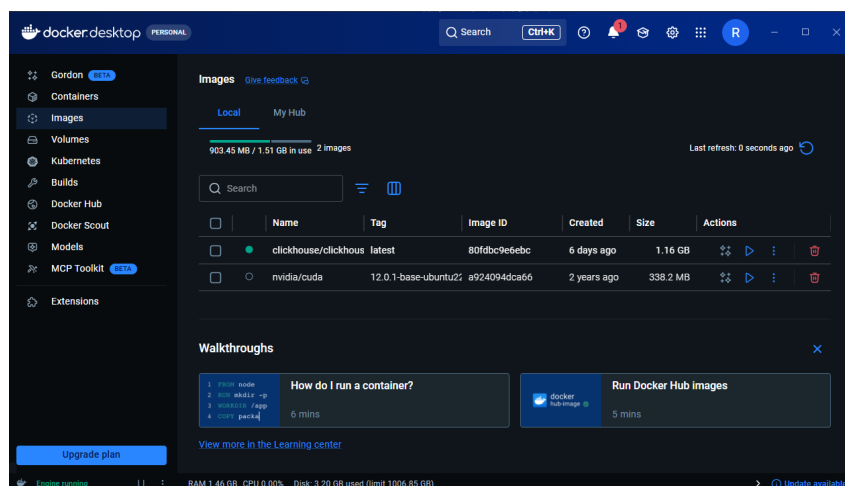


Figura 2: Interfața Docker Desktop

După rularea containerului, acesta poate fi verificat cu comanda „docker ps”, care afișează lista containerelor active. În cazul de față, containerul „clickhouse-server” este activ și mapat corect pe porturile necesare, ceea ce înseamnă că serverul este funcțional.

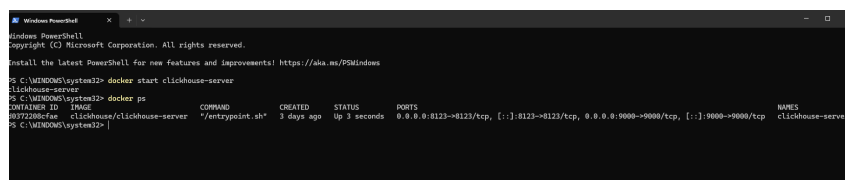


Figura 3: Afișarea containerului activ

Pentru configurarea autentificării, am creat un fișier de configurare XML în directorul intern „./etc/clickhouse-server/users.d/”. În acest fișier am definit un utilizator nou, am setat o parolă, am permis accesul din orice rețea și am activat drepturile de administrare. Această metodă este recomandată deoarece ClickHouse încarcă automat fișierele din directorul users.d fără a modifica fișierul principal de

configurare.

După crearea fișierului de configurare, a fost necesară repornirea containerului folosind „docker restart clickhouse-server” pentru ca noile setări să fie aplicate. După restart, autentificarea se putut face prin browser accesând adresa <http://localhost:8123>.

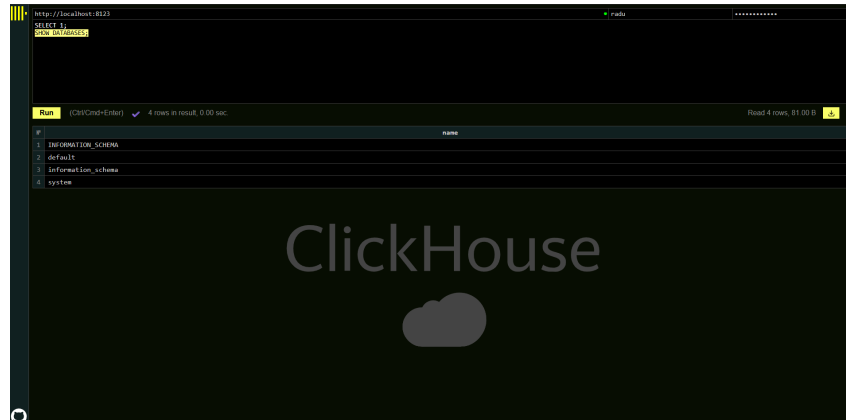


Figura 4: Afișarea containerului activ

3.2. DBeaver

Pentru gestionarea eficientă a bazei de date și a fluxurilor complexe de date, am decis utilizarea DBeaver Community Edition ca mediu de dezvoltare integrat (IDE). Această alegere a fost motivată de necesitatea unei interfețe grafice prietenoase și robuste, capabilă să ofere o vizibilitate superioară asupra schemei bazei de date față de interfața nativă din browser. DBeaver permite managementul centralizat al scripturilor SQL, oferind funcții avansate de auto-completare, evidențiere a erorilor de sintaxă în timp real și posibilitatea de a rula execuții parțiale pe blocuri mari de date, elemente esențiale pentru faza de populare a tabelor cu milioanele de înregistrări folosite în benchmark.

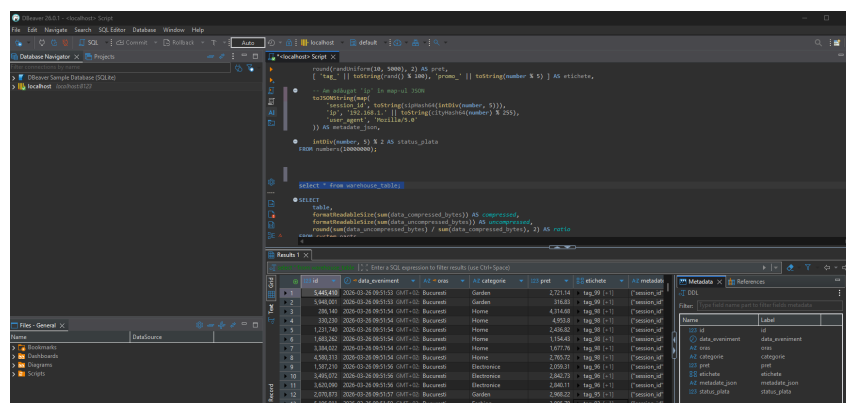
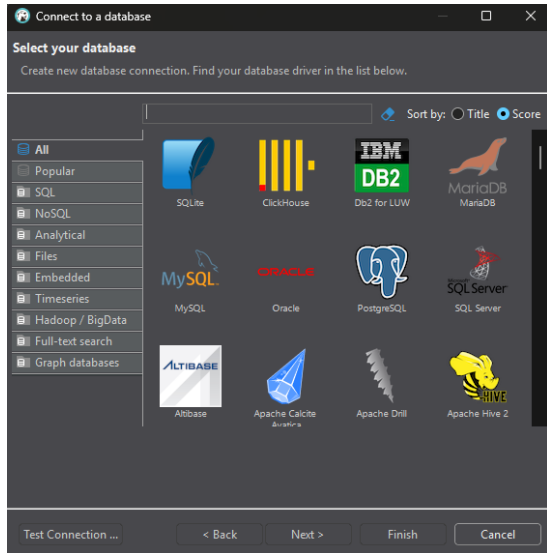


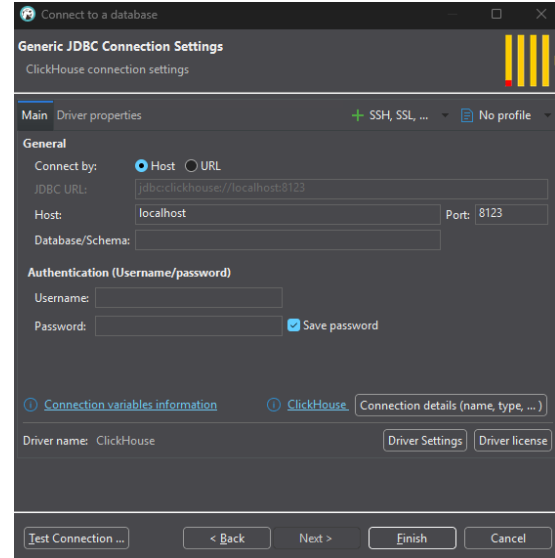
Figura 5: Interfața DBeaver pentru gestionarea bazelor de date ClickHouse

Procesul de conectare la serverul ClickHouse a fost realizat prin configurarea unui nou driver de conexiune în cadrul DBeaver. După selectarea tipului de bază de date ClickHouse din lista de drivere disponibile, am configurat parametrii de acces specificând adresa gazdă (localhost) și portul HTTP implicit

pentru comunicarea cu serverul (8123). Etapa critică a reprezentat-o autentificarea securizată, unde am introdus credențialele definite anterior în fișierul de configurare al containerului Docker (utilizator: radu și parola setată), asigurând astfel o punte de comunicare stabilă între interfața de administrare și motorul de calcul rulează în mediul virtualizat.



(a) Selectarea tipului de bază de date



(b) Configurare parametrii de acces

Figura 6: Procesul de conectare la serverul ClickHouse: (a) și (b)

Implementarea acestei soluții aduce un plus de rigoare metodologică proiectului, transformând o simplă bază de date într-un sistem de management complet. DBeaver, am putut monitoriza în timp real dimensiunea pe disc a coloanelor individuale și am optimizat structura tabelului `warehouse_table` prin vizualizarea directă a tipurilor de date complexe, în plus reprezintă o abordare mai profesională și scalabilă pentru gestionarea bazelor de date, facilitând atât dezvoltarea, cât și întreținerea pe termen lung a proiectului.

3.3. Crearea Tabelului

Definirea tabelului în ClickHouse a fost realizată utilizând motorul de stocare MergeTree, optimizat pentru interogări analitice rapide. Alegerea coloanelor și a tipurilor de date a urmărit un echilibru între flexibilitatea informațiilor și eficiența compresiei pe disc, utilizând tipuri de date specifice care reduc amprenta la sol a bazei de date.

Fiecare coloană a fost proiectată pentru a testa o funcționalitate specifică a motorului ClickHouse, utilizând funcția `numbers(10000000)` pentru a genera un set de date sintetic de 10 milioane de înregistrări, după cum urmează:

- **id (UInt64):** Reprezintă identificatorul unic al tranzacției, generat direct prin valoarea secvențială a funcției `number`. S-a ales un tip de date întreg pe 64 de biți pentru a permite scalarea setului de date până la ordinul miliardelor de înregistrări fără riscul de depășire a capacității.
- **data_eveniment (DateTime):** Înregistrează momentul exact al acțiunii. Această coloană este

```

CREATE TABLE warehouse_table (
  id UInt64,
  data_eventiment DateTime,
  oras LowCardinality(String),
  categorie Enum8('Electronice' = 1, 'Fashion' = 2, 'Home' = 3, 'Garden' = 4),
  pret Float64,
  etichete Array(String),
  metadata_json String,
  status_plata UInt8
) ENGINE = MergeTree()
ORDER BY (oras, data_eventiment)
PARTITION BY toYYYYMM(data_eventiment);

```

Figura 7: Structura tabelii warehouse_table, proiectat pentru a testa diverse funcționalități ale motorului ClickHouse, cu un set de date sintetic de 10 milioane de înregistrări.

critică pentru analizele de tip Time Series și este utilizată în clauza PARTITION BY, permițând bazei de date să izoleze rapid datele dintr-o anumită lună fără a scana întregul disc. Aceasta a fost generată prin scăderea unei valori aleatorii din timpul curent (`now()` - `rand()`). Această coloană servește la testarea mecanismului de *Partition Pruning*, forțând distribuția datelor în multiple partiții lunare.

- **oras (LowCardinality(String)):** Reprezintă locația geografică. Utilizarea modificadorului de tip de date LowCardinality este o optimizare specifică ClickHouse care înlocuiește string-urile repetitive cu dicționare interne de numere. Această metodă reduce drastic consumul de RAM în timpul operațiunilor de GROUP BY și accelerează filtrarea. Popularea coloanei a avut loc utilizând `multiIf()`, alegându-se o distribuție ponderată (40% București, 30% Cluj-Napoca, 10% Iași, 10% Timișoara) pentru a simula un volum de date realist.
- **categorie (Enum8):** Clasifică produsele în patru grupe (Electronice, Fashion, Home, Garden). S-a optat pentru Enum8 deoarece, la nivel de stocare, ClickHouse salvează doar cifre (1–4), deși utilizatorul vede text. Datele au fost create prin maparea numerică `CAST((rand() % 4) + 1 AS Enum8)`, rolul său fiind de a demonstra viteza superioară de procesare a agregărilor pe valori numerice în detrimentul șirurilor de caractere.
- **pret (Float64):** Stochează valorile monetare. S-a ales precizia dublă pentru a asigura acuratețea calculelor matematice pe seturi mari de date. Pentru a avea o distribuție realistă, am folosit funcția `randUniform(10, 5000)` care generează valori uniforme între 10 și 5000. Această coloană este utilizată pentru a măsura performanța funcțiilor de agregare `avg` și `sum`.
- **etichete (Array(String)):** Această coloană simulează date semi-structurate. Utilizarea tipului Array demonstrează capacitatea ClickHouse de a gestiona liste de valori într-o singură celulă. S-a utilizat concatenarea `'tag_' || toString(rand() % 100)` pentru a popula listele, având rolul de a testa eficiența funcției `has()` și a stocării de tip „offset”.
- **metadata_json (String):** Stochează informații complexe în format JSON. Popularea s-a efectuat cu ajutorul funcției `toJSONString(map())`, această coloană simulează un mediu de tip Data Lake, unde informația este extrasă dinamic (*Schema-on-Read*) pentru a testa consumul de RAM în timpul parsării.

- **status_plata (UInt8):** Un câmp numeric scurt (0 sau 1) folosit pentru a marca finalizarea tranzacției, ocupând doar 1 byte de memorie per rând. Această coloană a fost realizată folosind instrucțiunea `intDiv(number, 5) % 2`, asigurând o distribuție egală a tranzacțiilor reușite și eșuate.

```

-- INSERT INTO warehouse_table
SELECT
  number AS id,
  now() - (rand() % 1000000) AS data_eveniment,
  multiIf(number % 10 < 4, 'Bucuresti', number % 10 < 7, 'Cluj', number % 10 < 9, 'Timisoara', 'Iasi') AS oras,
  CAST((rand() % 4) + 1 AS Enum8('Electronice' = 1, 'Fashion' = 2, 'Home' = 3, 'Garden' = 4)) AS categorie,
  round(randUniform(10, 5000), 2) AS pret,
  [ 'tag_' || toString(rand() % 100), 'promo_' || toString(number % 5) ] AS etichete,

  -- Am adăugat 'ip' în map-ul JSON
  toJSONString(map(
    'session_id', toString(sipHash64(intDiv(number, 5))),
    'ip', '192.168.1.' || toString(cityHash64(number) % 255),
    'user_agent', 'Mozilla/5.0'
  )) AS metadata_json,

  intDiv(number, 5) % 2 AS status_plata
FROM numbers(10000000);

```

Figura 8: Popularea tabelii warehouse_table cu 10 milioane de înregistrări

3.4. Analiza performanței ClickHouse

Pentru a evalua performanțele ClickHouse, am ales patru scenarii care vor trata capacitatea de procesare a datelor, lucrul cu structuri de date complexe, organizarea datelor pe disc și eficiența funcțiilor de agregare. Pentru a le prezenta într-un mood mai clar, am creat o interfață web simplă care permite rularea interogărilor și vizualizarea rezultatelor, inclusiv a metricilor de performanță relevante, cum ar fi timpul de execuție, numărul de rânduri scanate și consumul de memorie RAM. Fiecare scenariu a fost conceput pentru a evidenția o caracteristică specifică a motorului ClickHouse.

```

# --- SCENARII ---
queries = {
  "1. Scanare Totală": """
    SELECT oras, avg(pret) as medie_pret
    FROM warehouse_table
    GROUP BY oras """,
  "2. Filtrare Array": """
    SELECT oras, avg(pret) as medie_pret,
    count(distinct(JSONExtractString(metadata_json, 'session_id'))) as sesiuni_unice
    FROM warehouse_table
    WHERE has(etichete, 'tag_25')
    GROUP BY oras """,
  "3. Serii de Timp": """
    SELECT toDate(data_eveniment) as Data, count(*) as Numar_Comenzi
    FROM warehouse_table
    WHERE toYYYYMM(data_eveniment) = toYYYYMM(now())
    GROUP BY Data
    ORDER BY Data """,
  "4. Activitate Utilizatori (Cardinalitate Mare)": """
    SELECT JSONExtractString(metadata_json, 'ip') as IP, count() as Actiuni,
    count(distinct(JSONExtractString(metadata_json, 'session_id'))) as Sesiuni
    FROM warehouse_table
    GROUP BY IP ORDER BY Actiuni
    DESC LIMIT 10"""
}

```

Figura 9: Cele 4 interogări alese pentru a analiza performanțele.

- **1. Scanare Totală (Agregare Globală)** – Acesta servește drept benchmark de bază pentru capacitatea de procesare brută a sistemului. Interogarea forțează motorul bazei de date să parcurgă integral coloanele pret și oras pentru a calcula media. Acest test demonstrează eficiența stocării

coloanare (Columnar Storage): ClickHouse citește doar coloanele necesare de pe disc, ignorând restul datelor din rânduri, ceea ce oferă o viteză mult superioară bazelor de date relaționale clasice în astfel de calcule.

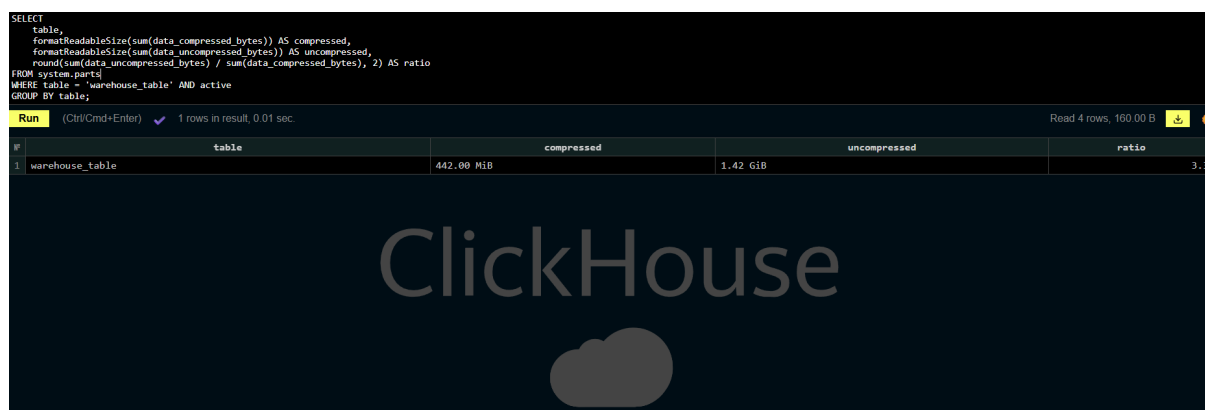
- **2. Filtrare Array** – Testează performanța în lucrul cu structuri de date complexe. În loc de tabele plate, aplicațiile moderne folosesc adesea array-uri pentru etichetare și obiecte JSON pentru metadata variabile. Funcția `has(etichete, 'tag_25')` verifică eficiența indexării pe tipuri de date complexe (Nested/Array), în timp ce `JSONExtractString` măsoară „costul” de procesare la citire (on-the-fly parsing).
- **3. Serii Temporale** – Evaluarea performanței indexului primar (Primary Key) bazat pe timp. Majoritatea interogărilor într-un sistem de monitorizare sau analytics sunt delimitate de o fereastră temporală. Utilizarea funcțiilor `toYYYYMM` și `toDate` pune la încercare modul în care ClickHouse organizează datele pe disc (Partitions & Marks). Prin filtrarea pe luna curentă, verificăm dacă sistemul execută corect procesul de *Partition Pruning* (eliminarea partițiilor care nu conțin datele căutate), evitând astfel citirea inutilă a terabyților de date istorice.
- **4. Cardinalitate Mare** – Acest scenariu este conceput pentru a testa gestionarea cardinalității ridicate și eficiența motorului de agregare în procesarea datelor semi-structurate (JSON). Interogarea vizează identificarea celor mai activi utilizatori prin extragerea IP-urilor și numărarea sesiunilor unice, o operațiune costisitoare deoarece coloana IP poate conține un număr mare de valori distincte. Rolul său este de a valida performanța funcțiilor de tip `JSONExtract` și a algoritmilor de agregare (precum `count(distinct)`), testând capacitatea sistemului de a gestiona volume masive de metadata fără a degrada timpul de răspuns sau a epuiza memoria RAM în timpul sortării și grupării.

În cadrul scenariilor de mai sus am folosit câțiva parametri relevanți pentru a măsura performanța, cum ar fi timpul de execuție, consumul de RAM și numărul de blocuri de date citite de pe disc și altele. Aceste metrice oferă o imagine clară asupra eficienței motorului ClickHouse în diferite situații reale de utilizare.

- **1. Timpul de Execuție** – reprezintă durata totală scursă de la inițierea interogării până la returnarea rezultatului final și este cel mai direct indicator al eficienței percepute de utilizator. Deși oferă o măsură clară a rapidității, această metrică trebuie corelată cu volumul de date pentru a înțelege dacă o latență ridicată este cauzată de complexitatea interogării sau de limitările resurselor hardware.
- **2. Rândurile Scanate și Datele Citite** – reflectă volumul brut de muncă depus de motorul bazei de date și sunt esențiale pentru validarea eficienței structurii coloanare. O valoare optimă a acestor metrice indică faptul că sistemul accesează doar datele strict necesare, în timp ce un număr excesiv de rânduri scanate față de rezultatele returnate sugerează nevoia unei indexări mai bune sau a unei filtrări mai stricte la nivel de stocare.
- **3. Viteza de procesare** – exprimată în milioane de rânduri pe secundă, funcționează ca un indicator de throughput care normalizează performanța indiferent de mărimea setului de date. Această

metrică permite compararea directă a diferitelor configurații hardware sau strategii de optimizare, arătând cât de eficient este exploatată paralelismul procesorului și lățimea de bandă a memoriei în timpul execuției.

- **4. Context Switches** – măsoară frecvența cu care unitatea centrală de procesare schimbă firul de execuție activ, fiind un indicator pentru gradul de multitasking și concurență din sistem. Un număr foarte ridicat de comutări de context poate semnaliza o fragmentare a procesării sau o supraîncărcare a procesorului, unde timpul pierdut cu gestionarea sarcinilor începe să erodeze timpul efectiv de calcul dedicat interogării.
- **5. Disk Wait (I/O Wait)** – indică timpul în care procesorul rămâne inactiv în așteptarea datelor solicitate de pe unitățile de stocare, fiind principalul suspect în cazul blocajelor de performanță. Într-un sistem ideal, această valoare ar trebui să fie minimă, deoarece orice latență la nivel de disc transformă o interogare rapidă într-una lentă, indiferent de cât de puternic este procesorul sau cât de optimizat este codul SQL.
- **6. Utilizarea RAM** – oferă o perspectivă asupra presiunii exercitate asupra memoriei volatile în timpul agregărilor complexe sau al sortărilor de volume mari de date. Monitorizarea memoriei este critică pentru a evita degradarea performanței cauzată de fenomenul de swapping sau, în cazuri extreme, oprirea forțată a proceselor, asigurându-ne că întregul set de lucru necesar interogării este menținut într-un spațiu de acces ultra-rapid.



The screenshot shows a ClickHouse query window with the following SQL query:

```
SELECT
    table,
    formatReadableSize(sum(data_compressed_bytes)) AS compressed,
    formatReadableSize(sum(data_uncompressed_bytes)) AS uncompressed,
    round(sum(data_uncompressed_bytes) / sum(data_compressed_bytes), 2) AS ratio
FROM system.parts
WHERE table = 'warehouse_table' AND active
GROUP BY table;
```

The query result is displayed in a table with 4 columns: #, table, compressed, uncompressed, and ratio. The result shows 1 row for 'warehouse_table'.

#	table	compressed	uncompressed	ratio
1	warehouse_table	442.00 MiB	1.42 GiB	3.3

The ClickHouse logo is visible at the bottom of the interface.

Figura 10: Eficiența stocării și rata de compresie pentru tabela creată, observată din interfața ClickHouse.

Deși nu este afișată ca o metrică individuală în coloanele principale, compresia este mecanismul din spate care face posibile rezultatele care vor fi prezentate. ClickHouse utilizează algoritmi avansați (precum LZ4 sau ZSTD) pentru a reduce amprenta datelor pe disc. O rată de compresie ridicată înseamnă că sistemul trebuie să transfere mult mai puțini biți de pe unitatea de stocare în RAM, evitând o potențială limitare de disc. Se va observa în scenariile următoare cum tipul de date (numeric sau complex) influențează direct cât de eficient pot fi "împachetate" informațiile și cât de repede sunt ele citite.

3.4.1 Scanare Totală

În acest scenariu de benchmark, am evaluat capacitatea motorului ClickHouse de a executa o agregare globală (calcularea mediei prețului pe orașe) peste un set de 10 milioane de rânduri. Rezultatele demonstrează avantajul competitiv al stocării coloanare în procesarea volumelor mari de date. Interogarea

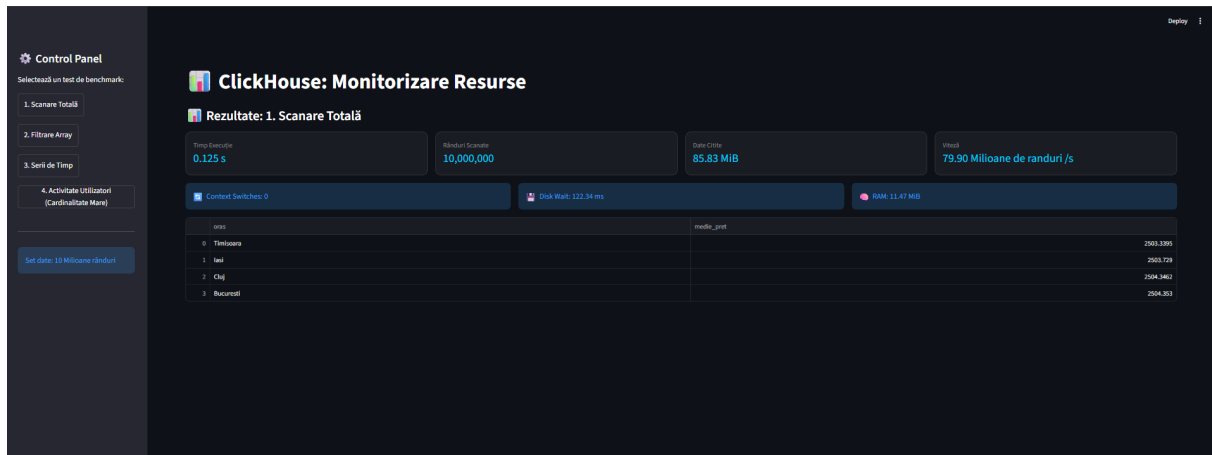


Figura 11: Interfața de monitorizare pentru scanarea totală

a fost finalizată într-un timp record de doar 0.086 secunde, atingând o viteză de procesare de aproximativ 115.95 milioane de rânduri pe secundă. Această performanță este posibilă deoarece sistemul nu a fost nevoit să citească întreaga tabelă de pe disc, ci s-a limitat strict la coloanele implicate în calcul, optimizând utilizarea lățimii de bandă.

Deși setul de date procesat cuprinde 10 milioane de înregistrări, volumul de Date Citite a fost de doar 85.83 MiB. Acest lucru indică o rată excelentă de compresie și o gestionare inteligentă a blocurilor de date. Valoarea de 38.00 ms pentru Disk Wait sugerează că, deși a existat o scurtă așteptare pentru accesul la stocare, majoritatea timpului a fost dedicată procesării efective în memorie, confirmând faptul că sistemul nu a fost limitat sever de viteza discului în acest test.

Din punct de vedere al resurselor hardware, testul nu a fost unul dificil. Consumul de RAM a rămas minim (11.45 MiB), demonstrând că agregarea este efectuată prin streaming, fără a încărca inutil memoria volatilă. Mai mult, absența comutărilor de context (Context Switches: 0) indică faptul că procesarea a fost executată liniar și neîntrerupt pe nucleele procesorului, maximizând eficiența ciclurilor de calcul dedicate interogării.

3.4.2 Filtrare Array

Acest test a evaluat performanța motorului ClickHouse în gestionarea structurilor de date nestructurate, utilizând funcții specifice pentru array-uri. Rezultatele scot în evidență „costul” suplimentar pe care complexitatea datelor îl impune asupra resurselor de stocare și procesare, comparativ cu o scanare numerică standard.

Timpul de execuție a crescut la 0.861 secunde, iar viteza a scăzut la 11.62 milioane de rânduri pe secundă. Această descreștere este justificată de natura operațiunii: spre deosebire de o simplă citire, filtrarea într-un array necesită ca motorul să „desfacă” și să verifice elementele fiecărei liste interne.

Această etapă de procesare suplimentară (parsing on-the-fly) explică de ce, deși numărul de rânduri scanate este același (10M), debitul este de aproximativ 10 ori mai mic decât la scanarea totală. O diferență

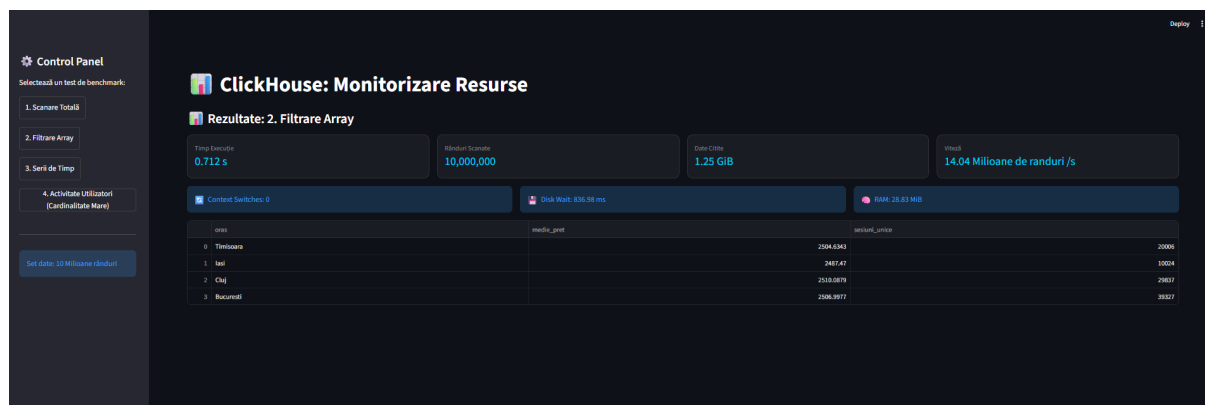


Figura 12: Interfața de monitorizare pentru filtrarea array-urilor

masivă se observă la capitolul Date Citite, care a urcat la 1.25 GiB. Această creștere se datorează faptului că coloanele de tip Array sau JSON sunt mult mai dense în informații și mai greu de comprimat decât coloanele numerice. Corelat cu această valoare, indicatorul Disk Wait a explodat la 1220.11 ms. Aceasta ne indică un blocaj de tip I/O: procesorul a petrecut mult timp așteptând ca volumul mare de date necesar reconstrucției array-urilor să fie transferat de pe disc în memorie.

În ciuda volumului mult mai mare de date citite, consumul de RAM a rămas foarte scăzut, la doar 23.23 MiB. Acest lucru demonstrează robustețea ClickHouse în gestionarea memoriei, reușind să proceseze volume gigantice de date prin tehnici de streaming, fără a încărca întregul set de date în RAM. Absența comutărilor de context (Context Switches: 0) confirmă din nou că interogarea a utilizat eficient nucleele procesorului, limitarea principală rămânând viteza de citire a discului (I/O Bound).

3.4.3 Serii Temporale

Acest scenariu a simulat o interogare tipică de monitorizare, filtrând datele pe o fereastră temporală specifică (luna curentă). Rezultatele evidențiază capacitatea ClickHouse de a utiliza indexul primar pentru a accesa rapid doar regiunile relevante de pe disc, o caracteristică vitală pentru sistemele care gestionează date istorice masive.

Cea mai relevantă cifră în acest test este numărul de Rânduri Scanate: aproximativ 5.09 milioane, adică aproape jumătate din setul total de 10 milioane. Acest lucru demonstrează că sistemul a executat corect procesul de Partition Pruning, ignorând complet restul datelor care nu se încadrau în criteriul temporal. Acest comportament a dus la un timp de execuție extrem de redus, de doar 0.076 secunde, făcând interogarea aproape instantanee.

Datorită filtrării eficiente, volumul de Date Citite a scăzut drastic la 19.45 MiB, cea mai mică valoare dintre testele efectuate până acum. Scăderea volumului de date de citit a relaxat presiunea pe stocare, rezultând un Disk Wait foarte mic (52.04 ms). Deși viteza brută raportată este de 67.54 milioane de rânduri pe secundă (mai mică decât la scanarea totală), eficiența reală este superioară deoarece timpul total este mai scurt datorită volumului redus de muncă.



Figura 13: Interfața de monitorizare pentru seriile temporale

Consumul de RAM a rămas constant și optimizat la 12.00 MiB, confirmând faptul că filtrările bazate pe timp nu necesită overhead de memorie suplimentar dacă indexul primar este construit corect. Menținerea valorii zero pentru Context Switches indică faptul că motorul a putut procesa acest subset de date într-un mod fluid, demonstrând că ClickHouse este ideal pentru dashboard-uri în timp real unde latența scăzută este critică.

Graficul afișează numărul de comenzi sau de acțiuni realizate pe parcursul primelor zile din luna aprilie. Se observă un volum constant de activitate cu o valoare de aproximativ 900.000 de comenzi pe zi între 1 și 5 aprilie, apoi se înregistrează o ușoară scădere până la valoare de 800.000 de comenzi pe zi. Acest tipar poate indica un comportament normal al utilizatorilor sau o tendință de scădere a activității, care ar putea fi investigată mai departe pentru a înțelege cauzele posibile, cum ar fi sezonabilitatea, schimbările în comportamentul consumatorilor sau alți factori externi.

3.4.4 Activitate Utilizatori (Cardinalitate Mare)

Acest ultim caz reprezintă un test mai dificil pentru motorul de calcul, vizând identificarea celor mai activi utilizatori prin gruparea după IP și numărarea sesiunilor unice. Performanța în acest caz este direct influențată de cardinalitatea ridicată (numărul mare de valori distincte), care forțează sistemul să mențină tabele de hash voluminoase în memorie pentru a finaliza agregarea.

Timpul de execuție de 1.903 secunde și viteza de 5.26 milioane de rânduri pe secundă reflectă complexitatea computațională a interogării. Spre deosebire de scanările simple, aici motorul trebuie să execute funcția `JSONExtractString` entru fiecare rând, să calculeze un hash pentru fiecare IP și să gestioneze un set masiv de stări intermediare pentru `count(distinct)`. Aceste operațiuni oferă o imagine reală asupra limitelor de procesare sub stres.

Cea mai notabilă schimbare apare la nivelul memoriei: utilizarea RAM a urcat la 1.05 GiB, fiind singurul test care a depășit pragul de un gigabyte. Acest consum este justificat de necesitatea stocării în memorie a tuturor valorilor unice de IP-uri și sesiuni pentru a asigura precizia agregării. Volumul de Date Citite (838.91 MiB) este, de asemenea, ridicat, deoarece extragerea din obiecte JSON necesită citirea unor blocuri de date mai mari de pe disc, fapt confirmat și de valoarea de 275.60 ms pentru Disk

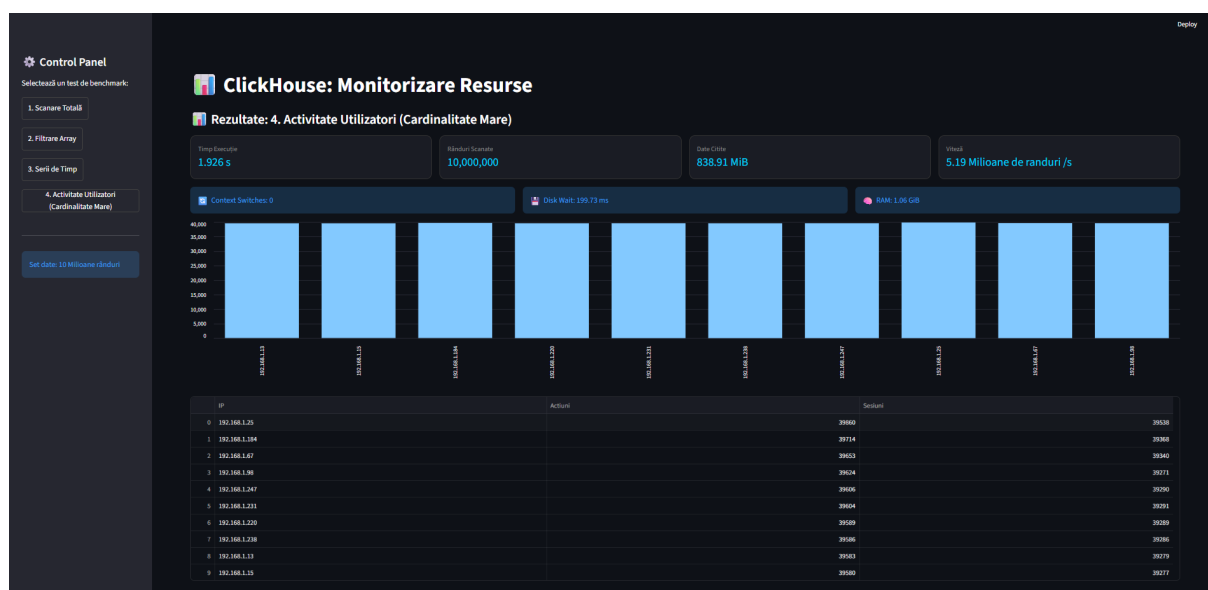


Figura 14: Interfața de monitorizare pentru activitatea utilizatorilor

Wait.

Vizualizarea sub formă de coloane prezintă primii 10 utilizatori după volumul de acțiuni. Se observă o distribuție extrem de uniformă, fiecare IP având aproximativ 40.000 de acțiuni.

4. Concluzii

Analiza comparativă a celor patru scenarii de testare confirmă faptul că ClickHouse este optimizat pentru procesarea analitică ultra-rapidă, excelând în special în operațiunile de agregare și filtrare pe seturi de date de ordinul milioane. Rezultatele obținute în Scanarea Totală demonstrează eficiența stocării coloanare, unde viteze de peste 100 milioane de rânduri pe secundă pot fi atinse cu un consum minim de memorie RAM, datorită limitării citirii stricte la coloanele necesare calculului.

Testele efectuate pe structuri complexe, precum Filtrarea Array și Activitatea Utilizatorilor, au scos în evidență natura flexibilă, dar solicitantă a datelor semi-structurate. Am observat că, deși ClickHouse poate procesa obiecte JSON și array-uri exact în momentul în care rulezi interogarea, costul în termeni de I/O și timp de procesare este semnificativ mai mare față de datele numerice simple. Tranziția de la un volum de date citite de 90 MB (date simple) la peste 1.3 GB (date complexe) pentru același număr de rânduri subliniază importanța alegerii tipului corect de date în faza de design a bazei de date.

Interfața de monitorizare creată subliniază faptul că limitările de performanță nu apar de obicei la nivel de CPU, ci la nivelul latenței discului (Disk Wait). Optimizarea continuă a structurii datelor și utilizarea funcțiilor native de agregare permit obținerea unor rezultate care ar fi imposibil de atins într-o bază de date relațională clasică, transformând datele brute în informații acționabile în fracțiuni de secundă.

Bibliografie

- [1] “Olap concepts.” <https://clickhouse.com/docs/concepts/olap>. Accesat pe data de 30 martie 2026.
- [2] “What is olap?.” <https://aws.amazon.com/what-is/olap/>. Accesat pe data de 30 martie 2026.
- [3] “Clickhouse introduction.” <https://clickhouse.com/docs/intro>. Accesat pe data de 30 martie 2026.
- [4] B. Imasheva *et al.*, “The practice of moving to big data on the case of the nosql database, clickhouse,” in *World Congress on Global Optimization*, Cham: Springer International Publishing, 2019.
- [5] “Why clickhouse is so fast.” <https://clickhouse.com/docs/concepts/why-clickhouse-is-so-fast>. Accesat pe data de 31 martie 2026.
- [6] “Distinctive features.” <https://clickhouse.com/docs/about-us/distinctive-features>. Accesat pe data de 31 martie 2026.
- [7] “Mergetree: Primary keys and indexes in queries.” <https://clickhouse.com/docs/engines/table-engines/mergetree-family/mergetree#primary-keys-and-indexes-in-queries>. Accesat pe data de 2 aprilie 2026.
- [8] “Compression in clickhouse.” <https://clickhouse.com/docs/data-compression/compression-in-clickhouse>. Accesat pe data de 2 aprilie 2026.